Python Terminology

A Beginner's Glossary

DCU International Foundation Year

Computer Science

A companion to the Python Reference Document. Each row explains one term in simple language. Use this as a reference while programming; examples are independent and may assume that variables have already been defined.

1 Programming fundamentals

The words used to describe programs and the tools used to write them.

Term	Simple meaning	Example or reminder
Program	A set of instructions that a computer can run.	A calculator or guessing game.
Programming	Writing instructions to solve a problem using a computer.	Writing a program to calculate an average.
Algorithm	A clear sequence of steps for solving a problem.	Read two numbers, add them, show the total.
Pseudocode	A way to plan an algorithm using plain words and code-like steps.	INPUT age; OUTPUT age + 1
Flowchart	A diagram showing the steps and decisions in an algorithm.	A diamond represents a decision.
Decomposition	Breaking a large problem into smaller, manageable parts.	Split a quiz into questions, scoring and results.
Abstraction	Focusing on the details needed to solve a problem.	A map shows roads without showing every tree.
Source code	The instructions written by a programmer.	The text in a Python .py file.
Script	A file of code that can be run to perform a task.	quiz.py
Interpreter	Software that runs Python code.	The Python interpreter executes your program.
Execution	Running the instructions in a program.	Click Run to execute your code.
IDE	An application with tools for writing, running and debugging code.	IDE means integrated development environment.
Editor	A tool for writing and changing source code.	The area where you type your program.
Console	A text area used for input, output or commands.	Printed results may appear in the console.
Interactive shell	An environment where you enter Python code and see results immediately.	The Python prompt is often >>>.
Syntax	The rules for writing valid code.	An if statement needs a colon.
Statement	An instruction or group of instructions that Python executes.	score = 10
Expression	Code that is evaluated to produce a value.	3 + 4 produces 7.
Comment	Text for a human reader that Python does not execute.	# Calculate the total
Indentation	Spaces at the start of a line that show which block it belongs to.	Use four spaces inside an if or loop.
Block	A group of statements that belong together.	The indented statements inside a loop.
Keyword	A word reserved by Python for a particular purpose.	if, while and return are keywords.
Case-sensitive	Treating uppercase and lowercase letters as different.	score and Score are different names.

2 Variables and data types

These terms describe the values a program stores and works with.

Term	Simple meaning	Example or reminder
Data	Information used or produced by a program.	Names, marks and temperatures.
Value	A particular piece of data.	42 or "Hello"
Variable	A name that refers to a value; it can be assigned a different value.	score = 10
Identifier	A name chosen for something in a program.	total_cost is a variable identifier.
Assignment	Giving a name a value using =.	age = 18
Initialisation	Giving a variable its starting value.	total = 0 before adding values.
Reassignment	Giving an existing variable a new value.	score = 20 after score = 10
Constant	A value intended to stay unchanged. Python does not enforce this for ordinary names.	MAX_SCORE = 100
Literal	A value written directly in source code.	12, 3.5 and "cat" are literals.
Data type	The kind of value, which affects the operations it supports.	int, float, str and bool
Integer (int)	A whole number, including zero and negative numbers.	-3, 0, 25
Float (float)	A floating-point number, commonly used for decimals; many values are stored approximately.	3.5 or -0.25
String (str)	A sequence of characters used to represent text.	"Hello" or "123"
Character	One symbol in text. Python represents it as a string of length one.	"A", "7" or "!"
Boolean (bool)	A value that is either True or False.	finished = False
None	A special value commonly used to mean no value or no result.	result = None
Type conversion	Changing a value from one data type to another.	int("12") produces 12.
Input	Data entering a program. input() returns typed text as a string.	name = input("Name: ")
Output	Information that a program produces.	print("Hello") displays text.
Mutable	Able to be changed after it has been created.	A list can have an item added.
Immutable	Unable to be changed after it has been created.	Strings and tuples are immutable.
Dynamic typing	A name can refer to values of different types at different times.	x = 3, then x = "hello"

3 Operators decisions and loops

These terms explain calculations, choices and repetition.

Term	Simple meaning	Example or reminder
Operator	A symbol or word that performs an operation on values.	+ adds; and combines conditions.
Operand	A value used by an operator.	In 4 + 2, the operands are 4 and 2.
Arithmetic operator	An operator used for a numerical calculation.	+ for addition; * for multiplication.
Division	Dividing one number by another using /.	7 / 2 produces 3.5.
Floor division	Dividing and rounding the result down using //.	7 // 2 is 3; -7 // 2 is -4.
Modulo	Finding the remainder using %.	7 % 2 produces 1.
Exponentiation	Raising a number to a power using **.	2 ** 3 produces 8.
Precedence	The rules deciding which operations happen first.	Multiplication happens before addition.
Comparison	Checking the relationship between values.	age >= 18 produces True or False.
Equality	Having the same value; tested with ==.	5 == 5 is True. = assigns a value.
Boolean logic	Combining or reversing truth values using and, or and not.	True and False is False.
AND	True only when both conditions are true.	True and False is False.
OR	True when at least one condition is true.	True or False is True.
NOT	Reverses a Boolean value.	not True is False.
Condition	An expression checked to decide what should happen.	score >= 50
Truth table	A table showing every input combination and its Boolean result.	True or False gives True.
Control flow	The order in which a program executes its instructions.	A decision can change which statements run.
Sequence	Executing instructions one after another in order.	Read input, calculate, then print.
Selection	Choosing which instructions run based on a condition.	An if / elif / else structure.
Iteration	Repetition; also one pass through a loop.	A loop with five iterations repeats five times.
For loop	A loop that visits each item in an iterable.	for name in names:
While loop	A loop that repeats while its condition is true.	while attempts < 3:
Infinite loop	A loop that keeps running without reaching a stopping point.	A while condition that never becomes false.
Nested loop	A loop inside another loop.	An outer loop for rows and an inner loop for columns.
Counter	A variable used to count events or repetitions.	count += 1
Accumulator	A variable used to build up a result.	total += price
Break	A statement that exits the nearest enclosing loop.	break stops that loop immediately.
Continue	A statement that skips to the next iteration of the nearest loop.	continue skips the remaining loop body.

4 Functions and reusable code

Functions let you name a task, supply inputs and reuse its instructions.

Term	Simple meaning	Example or reminder
Function	A reusable block of code that performs a task.	len("cat") calls the len function.
Definition	Code that creates a function and describes what it does.	def greet(name):
Function call	An instruction to run a function.	greet("Alex")
Parameter	A name in a function definition that receives an argument.	name in def greet(name):
Argument	A value supplied when calling a function.	"Alex" in greet("Alex")
Positional argument	An argument matched to a parameter by its position.	area(4, 3)
Keyword argument	An argument supplied using its parameter name.	area(width=4, height=3)
Default value	A parameter value used when that argument is not supplied.	def greet(name="Guest"):
Return value	The result a function sends back to its caller.	return width * height
Return statement	A statement that ends the current function call and optionally sends back a value.	return total
Print	A function that displays information; it does not return the displayed text.	print(5) displays 5 and returns None.
Scope	The part of a program where a name can be used.	A name created inside a function is usually local.
Local variable	A variable defined within a function, for use in that function.	A temporary total inside calculate().
Global variable	A variable defined at module level, outside functions.	A setting near the top of a file.
Built-in function	A function available without importing a module.	print(), input() and len()
Method	A function accessed through an object or class.	names.append("Alex")
Docstring	A string at the start of a function, class or module that documents it.	A short description just after def.
Module	A unit of Python code that can be imported, often a .py file.	import random
Import	Making a module or its contents available to your code.	from math import sqrt
Package	A way to organise related Python modules.	A collection of modules under one package name.
Library	A collection of reusable code for particular tasks.	Python's standard library includes math.
Standard library	Modules supplied with a normal Python installation.	random, math and pathlib
Recursion	A function calling itself to solve a smaller version of a problem.	It needs a stopping case.

5 Strings and collections

Collections hold multiple items. The examples here show how their terminology is used.

Term	Simple meaning	Example or reminder
Collection	An object that groups multiple items together.	A list, tuple, dictionary or set.
Element	An individual item in a collection.	20 is an element of [10, 20, 30].
List	An ordered, mutable collection that can contain duplicates.	[10, 20, 20]
Tuple	An ordered, immutable collection.	(3, 4)
Dictionary	A collection that connects unique keys to values.	{"name": "Alex"}
Key	The label used to look up a dictionary value.	"name" in {"name": "Alex"}
Key-value pair	One dictionary entry: a key together with its value.	"name": "Alex"
Set	A collection of unique items with no positional indexing.	{1, 2, 3}
Index	The position number of an item in a sequence.	names[0] selects the first item.
Zero-based indexing	Numbering positions from zero rather than one.	A three-item list has indexes 0, 1 and 2.
Negative index	An index that counts backwards from the end.	names[-1] selects the last item.
Slice	A portion of a sequence selected by a range of positions.	text[1:4] includes indexes 1, 2 and 3.
Length	The number of items in a collection or characters in a string.	len("cat") is 3.
Iterable	An object whose items can be visited by a loop.	Lists, strings and range objects are iterable.
Range	An object representing a sequence of integers with an excluded stop value.	range(3) provides 0, 1 and 2.
Membership	Whether a collection contains a value.	"red" in colours
Concatenation	Joining sequences together.	"Hi " + "Alex" produces "Hi Alex".
String formatting	Building text that includes values in a chosen format.	f"Price: {price:.2f}"
Escape sequence	A backslash sequence representing a special character in a string.	\n means a newline; \t means a tab.
Append	Add one item to the end of a list.	names.append("Alex")
In-place change	Changing the existing object rather than making a replacement.	names.sort() changes names itself.
Copy	A separate object made using the contents of another.	b = a.copy() copies a list; nested items remain shared.
Alias	Another name referring to the same object.	After b = a, both names refer to the same list.

6 Files errors and testing

Use these terms when saving information, finding mistakes and checking programs.

Term	Simple meaning	Example or reminder
File	Named data saved on a storage device.	notes.txt
File path	The location of a file or folder.	data/marks.txt
Working directory	The folder used as the starting point for relative file paths.	open("notes.txt") starts from this folder.
File mode	A setting controlling how a file is opened.	"r" reads; "w" overwrites; "a" appends.
Encoding	Rules for representing text as bytes.	encoding="utf-8"
Context manager	A mechanism that manages setup and cleanup around a block.	with open(...) closes the file afterwards.
Bug	A mistake that makes a program behave incorrectly.	Using subtraction when addition was intended.
Debugging	Finding and fixing mistakes in a program.	Inspect variables to find the wrong calculation.
Syntax error	Code that breaks Python's writing rules and cannot be parsed.	A missing colon after if.
Indentation error	Incorrect spacing at the start of lines that prevents parsing.	An if body with no indentation.
Runtime error	An error that occurs while code is executing.	Dividing by zero.
Logic error	Code runs but produces the wrong behaviour or result.	Calculating an average using the wrong divisor.
Exception	An event signalling a problem during execution; code can handle it.	ValueError from int("hello").
Exception handling	Responding to an exception using try and except.	Ask for another input after an invalid number.
Traceback	An error report showing the calls and locations leading to an exception.	Read the final error message, then the code location.
TypeError	An exception caused by an operation receiving an unsuitable type.	"3" + 2
ValueError	An exception where the type fits but the value is unsuitable.	int("hello")
NameError	An exception caused by using a name that cannot be found.	Using score before defining it.
IndexError	An exception caused by an index outside a sequence's range.	Using index 5 in a three-item list.
KeyError	An exception caused by looking up a missing dictionary key.	student["age"] when "age" is absent.
Testing	Running checks to see whether a program behaves as intended.	Try several inputs and compare the results.
Test case	A specific input or situation with an expected result.	Input 2 and 3; expect a total of 5.
Boundary test	A check at or near the edge of an allowed range.	For ages 16 and above, test 15, 16 and 17.
Validation	Checking that input meets the program's rules.	Check that a mark is between 0 and 100.
Trace table	A table recording variable values as code runs step by step.	Record count and total after each loop iteration.

7 Objects and turtle graphics

A few object terms help explain turtle commands and other Python code.

Term	Simple meaning	Example or reminder
Object	A value in Python that has a type and associated data and behaviour.	A list or a turtle is an object.
Class	A definition describing a type of object and its behaviour.	turtle.Turtle is a class.
Instance	An individual object of a particular class.	pen = turtle.Turtle() creates a Turtle instance.
Attribute	A named value associated with an object or class, accessed using a dot.	student.name could hold a name.
Dot notation	Using a dot to access something belonging to an object or module.	pen.forward(100) or math.pi
State	The values describing an object at a particular moment.	A turtle's position, direction and pen colour.
Turtle	A drawing object that moves and can leave a line behind it.	pen = turtle.Turtle()
Canvas	The area in which graphics are drawn.	The turtle drawing window.
Coordinate	A number used to specify a position.	(100, 50) gives an x and a y coordinate.
Origin	The position where both coordinates are zero.	(0, 0), normally the centre in turtle.
X axis	The horizontal direction in a coordinate system.	Positive x moves to the right.
Y axis	The vertical direction in a coordinate system.	Positive y moves upwards in turtle.
Heading	The direction the turtle faces, measured in degrees.	By default, 0 is right and 90 is up.
Relative movement	Movement measured from the current position or direction.	forward(50) or left(90)
Absolute position	A target position given using the coordinate system.	goto(100, 50)
Pen state	Whether movement draws a line.	penup() stops drawing; pendown() resumes it.
Fill	Colouring the inside of a drawn shape.	Use begin_fill() before and end_fill() after.
Event	An action that a program can respond to.	A mouse click or a key press.
Callback	A function supplied for another part of a program to call when needed.	A function that runs when a key is pressed.